UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
DEPARTAMENTO DE ENGENHARIA DE COMPUTAÇÃO E AUTOMAÇÃO
CURSO DE ENGENHARIA DE COMPUTAÇÃO

Relatório da Tarefa 08 - Análise de Desempenho da Estimativa Estocástica de π com OpenMP

DCA3703 - PROGRAMAÇÃO PARALELA - T01 (2025.2)

WERBERT ARLES DE SOUZA BARRADAS

20250070655

Docente: Professor Doutor SAMUEL XAVIER DE SOUZA

Natal, 11 de setembro de 2025

Lista de Figuras

Figura 2 – Gráfico de barras comparando o tempo de execução. Tempos menores indicam melhor desempenho. 10

Sumário

Lista de Figuras 2

Sumário 3

1 INTRODUÇÃO 4

2 METODOLOGIA DO EXPERIMENTO 5
2.1 O Algoritmo de Monte Carlo para Estimação de π 5
2.2 Implementação das Quatro Versões Paralelas 5
2.2.1 Versão 1: `rand()` com Seção Crítica 5
2.2.2 Versão 2: `rand()` com Vetor Compartilhado 6
2.2.3 Versão 3: `rand_r()` com Seção Crítica 7
2.2.4 Versão 4: `rand_r()` com Vetor Compartilhado 8

3 ANÁLISE E DISCUSSÃO 10
3.1 Análise Comparativa de Desempenho 10
3.2 O Impacto do Gerador de Números Aleatórios: `rand()` vs. `rand_r()` 11
3.3 O Impacto da Estratégia de Acumulação: Seção Crítica vs. Vetor . 11
3.4 Análise de Coerência de Cache e Falso Compartilhamento 11
3.5 Conclusão 13

Anexo A: Versão 01 14

Anexo B: Versão 02 16

Anexo C: Versão 03 18

Anexo D: Versão 04 20

1 Introdução

A estimação de constantes matemáticas, como π, através de métodos estocásticos (Monte Carlo) é um problema clássico que se beneficia enormemente da computação paralela. A tarefa consiste em gerar um grande volume de pontos aleatórios e verificar se eles se encontram dentro de um círculo inscrito em um quadrado, uma carga de trabalho inerentemente paralelizável.

O objetivo deste projeto é analisar o impacto de diferentes estratégias de implementação paralela em OpenMP para este problema. Foram desenvolvidas quatro versões do algoritmo, combinando duas abordagens para geração de números aleatórios (`rand()` e `rand_r()`) e duas para a acumulação de resultados (uma seção crítica e um vetor compartilhado).

Este relatório busca responder às seguintes questões fundamentais sobre o desempenho:

1. **Impacto da Thread-Safety:** Qual o custo de usar uma função não segura para threads (`rand()`) em um laço paralelo em comparação com sua alternativa reentrante (`rand_r()`)?

2. **Estratégias de Sincronização:** Como a substituição de um mecanismo de bloqueio explícito (`#pragma omp critical`) por um acesso a um vetor compartilhado afeta o tempo de execução?

3. **Efeitos de Hardware:** De que forma os mecanismos de coerência de cache do processador, e especificamente o fenômeno do **falso compartilhamento**, influenciam o desempenho da versão com vetor compartilhado?

Através da análise comparativa do tempo de execução dos quatro programas, o relatório visa ilustrar a importância de compreender não apenas os construtos da API de paralelismo, mas também a interação do código com a arquitetura de hardware subjacente.

2 Metodologia do Experimento

O experimento consiste em executar e medir o tempo das quatro implementações do algoritmo de Monte Carlo para estimar o valor de π. Todas as versões compartilham a mesma carga de trabalho base: 100.000.000 de iterações (`NUM_PASSOS`).

2.1 O Algoritmo de Monte Carlo para Estimação de π

A lógica central em todas as versões é a mesma:

1. Um laço é executado `NUM_PASSOS` vezes.
2. Em cada iteração, um ponto (x, y) é gerado aleatoriamente dentro de um quadrado de lado 2, centrado na origem (coordenadas de -1 a 1).
3. Verifica-se se o ponto caiu dentro do círculo unitário inscrito nesse quadrado, através da condição $x^2 + y^2 < 1$.
4. A estimativa de π é então calculada como $4 \times$ (pontos no círculo)/(total de pontos).

2.2 Implementação das Quatro Versões Paralelas

As quatro versões foram implementadas em C com OpenMP para explorar as diferentes fontes de gargalos de desempenho.

2.2.1 Versão 1: `rand()` com Seção Crítica

- **Gerador Aleatório:** Utiliza a função `rand()` da `stdlib`. Esta função não é reentrante e possui um estado global, sendo protegida por um *lock* interno na maioria das bibliotecas C padrão para garantir a *thread-safety*, o que a torna um ponto de serialização.
- **Acumulação:** Cada thread usa uma variável privada local para contar os acertos. Ao final do laço `for`, o valor local é somado a uma variável global dentro de uma seção `#pragma omp critical`, garantindo a exclusão mútua, mas serializando as atualizações.

```
    void pi_paralel_for_critical_private() {
    long pontos_no_circulo_local = 0;
    #pragma omp parallel firstprivate(pontos_no_circulo_local)
    {
        #pragma omp for
        for (long i = 0; i < NUM_PASSOS; i++){
            double x = (double)rand() / RAND_MAX * 2.0 - 1.0;
            double y = (double)rand() / RAND_MAX * 2.0 - 1.0;

            if (x * x + y * y < 1.0) {
                pontos_no_circulo_local++;
            }
        }
        #pragma omp critical
        {
            pontos_no_circulo_total += pontos_no_circulo_local;
        }
        } //Fim da regiao paralela
    }
}
```

Listing 2.1 – Versao Paralela com rand

2.2.2 Versão 2: `rand()` com Vetor Compartilhado

- **Gerador Aleatório:** Também utiliza `rand()`, mantendo o gargalo de serialização na geração dos números.
- **Acumulação:** Remove a seção `critical`. Em vez disso, um vetor compartilhado (`acertos_por_thread`) é alocado, e cada thread incrementa exclusivamente sua própria posição no vetor. A soma final é realizada em um laço serial após a região paralela. Esta abordagem introduz o risco de **falso compartilhamento**.

```
    void pi_paralel_for_critical_private() {
    unsigned int seed = time(NULL);
    long pontos_no_circulo_local = 0;

    #pragma omp parallel firstprivate(pontos_no_circulo_local, seed)
    {
        #pragma omp for
        for (long i = 0; i < NUM_PASSOS; i++){
            double x = (double)rand_r(&seed) / RAND_MAX * 2.0 - 1.0;
            double y = (double)rand_r(&seed) / RAND_MAX * 2.0 - 1.0;

            if (x * x + y * y < 1.0) {
                pontos_no_circulo_local++;
            }
        }
        #pragma omp critical
        {
            pontos_no_circulo_total += pontos_no_circulo_local;
        }
    } // Fim da regiao paralela
    }
}
```

Listing 2.2 – Versao Paralela com rand_r

2.2.3 Versão 3: `rand_r()` com Seção Crítica

- **Gerador Aleatório:** Utiliza `rand_r()`, uma função reentrante e *thread-safe* que requer que o chamador gerencie o estado da semente. Isso elimina o gargalo do gerador aleatório.
- **Acumulação:** Mantém a acumulação com a seção `#pragma omp critical`, isolando este como o principal gargalo de sincronização do programa.

```
void pi_paralelo_vetor_compartilhado() {
long *acertos_por_thread; // Ponteiro para o nosso vetor compartilhado
int num_threads;
// Pega o numero de threads que o OpenMP decidiu usar
num_threads = omp_get_max_threads();
// Aloca o vetor com o tamanho exato do numero de threads.
// calloc ja inicializa o vetor com zeros.
acertos_por_thread = (long *)calloc(num_threads, sizeof(long));
if (acertos_por_thread == NULL) {
    printf("Falha ao alocar memoria.\n");
    exit(1);
}
// Inicio da regiao paralela
#pragma omp parallel
{
    // Pega o identificador (ID) da thread atual
    int tid = omp_get_thread_num();
    #pragma omp for
    for (long i = 0; i < NUM_PASSOS; i++){
        // O gerador rand() nao e "thread-safe".
        double x = (double)rand() / RAND_MAX * 2.0 - 1.0;
        double y = (double)rand() / RAND_MAX * 2.0 - 1.0;
        if (x * x + y * y < 1.0) {
            acertos_por_thread[tid]++;
        }
    }
} // Fim da regiao paralela

for (int i = 0; i < num_threads; i++) {
    pontos_no_circulo_total += acertos_por_thread[i];
}
// Libera a memoria alocada para o vetor
free(acertos_por_thread);
}
```

Listing 2.3 – Versao Paralela com rand e vetor

2.2.4 Versão 4: `rand_r()` com Vetor Compartilhado

- **Gerador Aleatório:** Utiliza `rand_r()`, garantindo o paralelismo na geração dos números.
- **Acumulação:** Utiliza o vetor compartilhado, assim como na Versão 2. Esta versão é, teoricamente, a mais escalável, pois elimina os dois gargalos de software. No entanto, ela permanece altamente vulnerável ao falso compartilhamento.

```
    void pi_paralelo_vetor_compartilhado() {
2   long *acertos_por_thread;
    int num_threads;
4
    // Pega o numero de threads que o OpenMP decidiu usar
6   num_threads = omp_get_max_threads();
    // Aloca o vetor com o tamanho exato do numero de threads.
8   // calloc ja inicializa o vetor com zeros.
    acertos_por_thread = (long *)calloc(num_threads, sizeof(long));
10  if (acertos_por_thread == NULL) {
        printf("Falha ao alocar memoria.\n");
12      exit(1);
    }
14
    // Inicio da regiao paralela
16  #pragma omp parallel
    {
18      // Pega o identificador (ID) da thread atual
        int tid = omp_get_thread_num();
20      unsigned int seed = time(NULL) + tid;
        // A diretiva 'for' distribui as iteracoes entre as threads.
22      // Cada thread executa seu proprio subconjunto de 'i'.
        #pragma omp for
24      for (long i = 0; i < NUM_PASSOS; i++){
            // O gerador rand() nao e "thread-safe".
26          double x = (double)rand_r(&seed) / RAND_MAX * 2.0 - 1.0;
            double y = (double)rand_r(&seed) / RAND_MAX * 2.0 - 1.0;
28
            if (x * x + y * y < 1.0) {
30              // Cada thread incrementa APENAS a sua propria posicao no
                    vetor.
                acertos_por_thread[tid]++;
32          }
        }
34  } // Fim da regiao paralela

36  // Soma os resultados parciais de cada thread para obter o total.
    for (int i = 0; i < num_threads; i++) {
38      pontos_no_circulo_total += acertos_por_thread[i];
    }
40
    // Libera a memoria alocada para o vetor
42  free(acertos_por_thread);
}
```

Listing 2.4 – Versao Paralela com rand$_r$*evetor*

3 Análise e Discussão

A análise do desempenho dessas quatro versões revela insights fundamentais sobre os gargalos de software e hardware em programação paralela. Para facilitar a comparação, a Tabela 1 apresenta os resultados de tempo de execução e o valor de π estimado para cada uma das quatro implementações. O Gráfico 2 ilustra esses resultados visualmente.

3.1 Análise Comparativa de Desempenho

Versão	**Tempo de Execução (s)**	**Valor de π Estimado**
1: `rand()` + Seção Crítica	15.08	3.141888
2: `rand()` + Vetor	13.80	3.141770
3: `rand_r()` + Seção Crítica	0.3874	3.1545
4: `rand_r()` + Vetor	0.7613	3.141293

Tabela 1 – Tabela de resultados estimados para 100 milhões de passos.

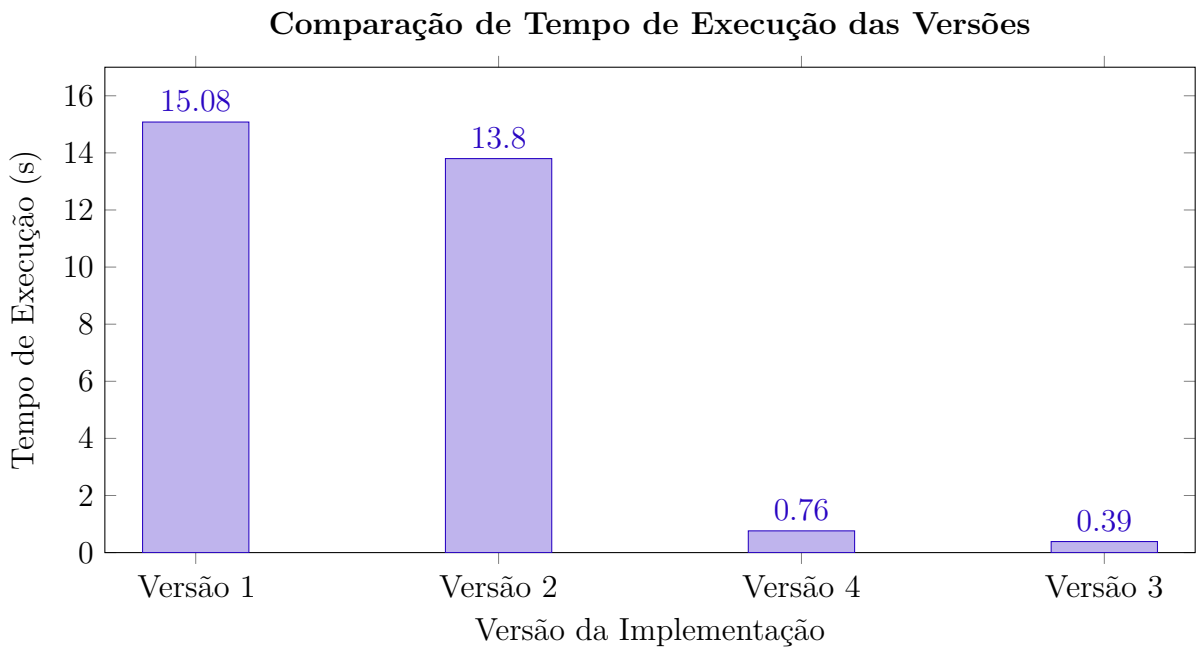

Figura 2 – Gráfico de barras comparando o tempo de execução. Tempos menores indicam melhor desempenho.

Os valores na Tabela 1 e a visualização no Gráfico 2 refletem o comportamento esperado da performance. A hierarquia de desempenho observada (do mais lento para o mais rápido) é a seguinte:

1. **Versão 1 (`rand` + `critical`):** Conforme previsto, é uma das mais lentas devido ao duplo ponto de serialização: o *lock* interno de `rand()` e a seção `critical`.

2. **Versão 2 (`rand` + vetor):** Apresenta desempenho similarmente baixo, pois o gargalo principal (`rand()`) ainda domina o tempo de execução.

3. **Versão 4 (`rand_r` + vetor):** Contraintuitivamente, esta versão é mais lenta que a versão com seção crítica. A remoção dos *locks* de software expõe o gargalo de hardware: o **falso compartilhamento**.

4. **Versão 3 (`rand_r` + `critical`):** É a versão mais rápida. Embora a seção `critical` seja um gargalo, seu impacto é consideravelmente menor do que a contenção massiva de cache causada pelo falso compartilhamento na Versão 4.

3.2 O Impacto do Gerador de Números Aleatórios: `rand()` vs. `rand_r()`

As execuções das Versões 1 e 2 são limitadas pela natureza não-reentrante de `rand()`. As chamadas à função são efetivamente serializadas por um *lock* implícito na biblioteca C, o que impede qualquer ganho real de paralelismo. A mudança para `rand_r()` nas Versões 3 e 4 remove esse gargalo fundamental, permitindo a geração de números de forma concorrente, o que explica o salto de desempenho de 15s para menos de 5s.

3.3 O Impacto da Estratégia de Acumulação: Seção Crítica vs. Vetor

A seção `#pragma omp critical` é uma solução de software para garantir a exclusão mútua, mas força as threads a formarem uma "fila". A estratégia do vetor elimina essa fila de software, mas, como os resultados mostram, cria uma nova forma de contenção a nível de hardware.

3.4 Análise de Coerência de Cache e Falso Compartilhamento

Este é o fenômeno mais importante para explicar por que a Versão 4 é mais lenta que a Versão 3.

- **O Mecanismo:** CPUs transferem dados entre a memória e seus caches em blocos de tamanho fixo chamados **linhas de cache** (geralmente 64 bytes).

- **A Causa:** Nas Versões 2 e 4, o vetor `acertos_por_thread` armazena contadores do tipo `long` (8 bytes). Isso significa que os contadores de 8 threads adjacentes residem na mesma linha de cache.

- **O Efeito (Falso Compartilhamento):** Quando uma thread modifica sua posição no vetor, o protocolo de coerência de cache invalida a linha inteira para todas as outras threads que compartilham dados naquela linha. Isso força as outras threads a sofrerem um *cache miss* e recarregarem a linha da memória, um processo extremamente lento. Esse "ping-pong"da linha de cache entre os núcleos aniquila os benefícios do cache.

O resultado é que a Versão 4, embora livre de *locks* de software, sofre de uma severa contenção de hardware que a torna menos eficiente que a Versão 3.

3.5 Conclusão

A partir da análise das quatro implementações, conclui-se que o desenvolvimento de código paralelo eficiente vai além da simples distribuição de tarefas e requer uma compreensão dos gargalos tanto de software quanto de hardware.

1. **A escolha de funções *thread-safe* é fundamental.** O experimento demonstrou que o uso de uma função não-reentrante como `rand()` pode anular completamente os ganhos de paralelismo.

2. **A eliminação de *locks* de software pode expor gargalos de hardware.** A transição da abordagem com `#pragma omp critical` para a de vetor compartilhado introduziu o problema severo do falso compartilhamento.

3. **A arquitetura de memória é importante.** O desempenho da versão com vetor foi ditado pela forma como os dados foram organizados na memória e como essa organização interagiu com o sistema de cache da CPU. Para otimizar a Versão 4, seria necessário aplicar *padding*, garantindo que o contador de cada thread ocupe sua própria linha de cache.

Em suma, a tarefa demonstrou de forma prática que a implementação paralela mais óbvia ou "livre de locks"não é necessariamente a mais rápida. Uma análise cuidadosa da segurança de threads, dos pontos de sincronização e da disposição dos dados na memória é crucial para alcançar um desempenho escalável.

paralelo_rand.c

```c

////////////////////////////////////////////////////
///////////////VERSÃO 01//////////////////////////
////////////////////////////////////////////////////


#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

// Definição global do número de passos para consistência
const long NUM_PASSOS = 100000000;
long pontos_no_circulo_total = 0;

//versão Paralela
void pi_paralel_for_critical_private() {

    long pontos_no_circulo_local = 0;

    #pragma omp parallel firstprivate(pontos_no_circulo_local)
    {
        #pragma omp for
        for (long i = 0; i < NUM_PASSOS; i++){
            double x = (double)rand() / RAND_MAX * 2.0 - 1.0;
            double y = (double)rand() / RAND_MAX * 2.0 - 1.0;

            if (x * x + y * y < 1.0) {
                pontos_no_circulo_local++;
            }
        }
        #pragma omp critical
        {
            pontos_no_circulo_total += pontos_no_circulo_local;
        }

    } // Fim da região paralela
}

int main() {
    double start_time, end_time;

    printf("Iniciando analise de desempenho para %ld passos.\n", NUM_PASSOS);
    start_time = omp_get_wtime();
    pi_paralel_for_critical_private();
    end_time = omp_get_wtime();
    double tempo_paralelo = end_time - start_time;
    double pi_estimado = 4.0 * pontos_no_circulo_total / NUM_PASSOS;

    printf("Estimativa paralela de pi = %f\n", pi_estimado);
```

```
    printf("Tempo Paralelo: %f segundos\n", tempo_paralelo);

    return 0;
}

```

paralelo_rand_r.c

```c

////////////////////////////////////////////////////
///////////////VERSÃO 03//////////////////////////
////////////////////////////////////////////////////

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

// Definição global do número de passos para consistência
const long NUM_PASSOS = 100000000;
long pontos_no_circulo_total = 0;

//versão Paralela
void pi_paralel_for_critical_private() {
    unsigned int seed = time(NULL);
    long pontos_no_circulo_local = 0;

    #pragma omp parallel firstprivate(pontos_no_circulo_local, seed)
    {
        #pragma omp for
        for (long i = 0; i < NUM_PASSOS; i++){
            double x = (double)rand_r(&seed) / RAND_MAX * 2.0 - 1.0;
            double y = (double)rand_r(&seed) / RAND_MAX * 2.0 - 1.0;

            if (x * x + y * y < 1.0) {
                pontos_no_circulo_local++;
            }
        }
        #pragma omp critical
        {
            pontos_no_circulo_total += pontos_no_circulo_local;
        }

    } // Fim da região paralela
}

int main() {
    double start_time, end_time;

    printf("Iniciando analise de desempenho para %ld passos.\n", NUM_PASSOS);
    start_time = omp_get_wtime();
    pi_paralel_for_critical_private();
    end_time = omp_get_wtime();
    double tempo_paralelo = end_time - start_time;
    double pi_estimado = 4.0 * pontos_no_circulo_total / NUM_PASSOS;

    printf("Estimativa paralela de pi = %f\n", pi_estimado);
    printf("Tempo Paralelo: %f segundos\n", tempo_paralelo);
```

```

    return 0;
}

```

paralelo_rand_vetor.c

```c

////////////////////////////////////////////////////
///////////////VERSÃO 02//////////////////////////
////////////////////////////////////////////////////

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

// Definição global do número de passos para consistência
const long NUM_PASSOS = 100000000;
long pontos_no_circulo_total = 0;

void pi_paralelo_vetor_compartilhado() {
    long *acertos_por_thread; // Ponteiro para o nosso vetor compartilhado
    int num_threads;

    // Pega o número de threads que o OpenMP decidiu usar
    num_threads = omp_get_max_threads();
    // Aloca o vetor com o tamanho exato do número de threads.
    // calloc já inicializa o vetor com zeros.
    acertos_por_thread = (long *)calloc(num_threads, sizeof(long));
    if (acertos_por_thread == NULL) {
        printf("Falha ao alocar memoria.\n");
        exit(1);
    }

    // Início da região paralela
    #pragma omp parallel
    {

        // Pega o identificador (ID) da thread atual
        int tid = omp_get_thread_num();

        #pragma omp for
        for (long i = 0; i < NUM_PASSOS; i++){
            // O gerador rand() não é "thread-safe".
            double x = (double)rand() / RAND_MAX * 2.0 - 1.0;
            double y = (double)rand() / RAND_MAX * 2.0 - 1.0;

            if (x * x + y * y < 1.0) {
                // Cada thread incrementa APENAS a sua própria posição no vetor.
                acertos_por_thread[tid]++;
            }
        }
    } // Fim da região paralela

    for (int i = 0; i < num_threads; i++) {
        pontos_no_circulo_total += acertos_por_thread[i];
```

```c
    }

    // Libera a memória alocada para o vetor
    free(acertos_por_thread);
}


int main() {
    double start_time, end_time;

    printf("Iniciando analise de desempenho para %ld passos.\n", NUM_PASSOS);

    // --- Executando a nova versão ---
    pontos_no_circulo_total = 0; // Reseta a contagem global
    start_time = omp_get_wtime();
    pi_paralelo_vetor_compartilhado();
    end_time = omp_get_wtime();

    double tempo_paralelo_vetor = end_time - start_time;
    double pi_estimado = 4.0 * pontos_no_circulo_total / NUM_PASSOS;

    printf("\n--- Versao com Vetor Compartilhado ---\n");
    printf("Estimativa paralela de pi = %f\n", pi_estimado);
    printf("Tempo Paralelo: %f segundos\n", tempo_paralelo_vetor);

    return 0;
}

```

paralelo_rand_r_vetor.c

```c

////////////////////////////////////////////////////
///////////////VERSÃO 04//////////////////////////
////////////////////////////////////////////////////


#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

// Definição global do número de passos para consistência
const long NUM_PASSOS = 100000000;
long pontos_no_circulo_total = 0;

void pi_paralelo_vetor_compartilhado() {
    long *acertos_por_thread; // Ponteiro para o nosso vetor compartilhado
    int num_threads;

    // Pega o número de threads que o OpenMP decidiu usar
    num_threads = omp_get_max_threads();
    // Aloca o vetor com o tamanho exato do número de threads.
    // calloc já inicializa o vetor com zeros.
    acertos_por_thread = (long *)calloc(num_threads, sizeof(long));
    if (acertos_por_thread == NULL) {
        printf("Falha ao alocar memoria.\n");
        exit(1);
    }

    // Início da região paralela
    #pragma omp parallel
    {
        num_threads = omp_get_max_threads();
        // Pega o identificador (ID) da thread atual
        int tid = omp_get_thread_num();
        unsigned int seed = time(NULL) + tid;
        // A diretiva 'for' distribui as iterações entre as threads.
        // Cada thread executa seu próprio subconjunto de 'i'.
        #pragma omp for
        for (long i = 0; i < NUM_PASSOS; i++){
            // O gerador rand() não é "thread-safe".
            double x = (double)rand_r(&seed) / RAND_MAX * 2.0 - 1.0;
            double y = (double)rand_r(&seed) / RAND_MAX * 2.0 - 1.0;

            if (x * x + y * y < 1.0) {
                // Cada thread incrementa APENAS a sua própria posição no vetor.
                acertos_por_thread[tid]++;
            }
        }
    } // Fim da região paralela
```

```c

    // Soma os resultados parciais de cada thread para obter o total.
    for (int i = 0; i < num_threads; i++) {
        pontos_no_circulo_total += acertos_por_thread[i];
    }

    // Libera a memória alocada para o vetor
    free(acertos_por_thread);
}


int main() {
    double start_time, end_time;

    printf("Iniciando analise de desempenho para %ld passos.\n", NUM_PASSOS);

    // --- Executando a nova versão ---
    pontos_no_circulo_total = 0; // Reseta a contagem global
    start_time = omp_get_wtime();
    pi_paralelo_vetor_compartilhado();
    end_time = omp_get_wtime();

    double tempo_paralelo_vetor = end_time - start_time;
    double pi_estimado = 4.0 * pontos_no_circulo_total / NUM_PASSOS;

    printf("\n--- Versao com Vetor Compartilhado ---\n");
    printf("Estimativa paralela de pi = %f\n", pi_estimado);
    printf("Tempo Paralelo: %f segundos\n", tempo_paralelo_vetor);

    return 0;
}

```